



NODE JS

Curso: Desarrollo en entorno servidor con Node.js UF1844

Introducción	1
Módulos en Node.js	2
Módulos Propios.	4
Módulos nativos fundamentales.	5
📁 fs (File System)	5
🧭 path	7
🌐 http	9
🕒 timers	12
¿Por qué son tan importantes?	23
Programación Asíncrona (El motor de Node)	24
Los callback	27
Las Promesas	30
Fetch(): Comunicación entre el Frontend y el Servidor	45
Ejercicios.	48
Estructura en proyectos de back y front.	49
Package.json	50
Ejercicio 1: Conectamos un botón al back.	53
Trabajamos en el back.	53
Trabajamos en el front.	60
Ejercicio 2: Conectando una API Externa.	63
Conclusión Node.js (UF1844)	73

Introducción

[Node](#) (o más correctamente: *Node.js*) es un entorno que trabaja en tiempo de ejecución, de código abierto, multi-plataforma, que permite a los desarrolladores crear toda clase de herramientas de **lado servidor** y aplicaciones en [JavaScript](#). La ejecución en tiempo real está pensada para usarse fuera del contexto de un explorador web (es decir, ejecutarse directamente en una computadora o sistema operativo de servidor).

Node.js es un entorno de ejecución de JavaScript que funciona fuera del navegador. Con Node, el **mismo lenguaje (JS)** se usa tanto en el **frontend** como en el **backend**. Esto simplifica el aprendizaje y permite compartir funciones o estructuras entre ambas partes.

Es de **código abierto y multiplataforma**. Las aplicaciones Node.js están escritas en JavaScript puro y se pueden ejecutar dentro del entorno Node.js en Windows, Linux, etc.

La característica más importante de NodeJS, y que ahora otra serie de lenguajes están aplicando, es la de **no ser bloqueante**. Es decir, si durante la ejecución de un programa hay partes que necesitan un tiempo para producirse la respuesta, NodeJS no detiene el hilo de ejecución del programa, esperando que esa parte acabe, sino que continúa procesando las siguientes instrucciones. Cuando el proceso lento termina, entonces realiza las instrucciones que fueran definidas para realizar con los resultados recibidos. NodeJS fue pensado desde el primer momento para potenciar los beneficios de la programación asíncrona.

Node incluye **NPM (Node Package Manager)**, un gestor de paquetes con millones de módulos reutilizables. Esto acelera el desarrollo: en lugar de escribir todo desde cero, se instalan librerías que ya resuelven tareas comunes (como Express, bcrypt, dotenv, etc.).

Desde una perspectiva de desarrollo de servidor web, Node tiene un gran número de ventajas:



Módulos en Node.js

Node tiene una extensa colección de módulos disponibles para la creación de programas, que resuelven la más variada gama de necesidades de desarrollo además también permite que los creamos nosotros como en React los componentes. Un **módulo** en Node.js es cualquier archivo .js que contiene funciones, objetos o clases reutilizables. Es como un componente en React, pero en el backend. Estos módulos hay que importarlos.

[Node.js](#) tiene dos formas de importar módulos.

Inma Contreras@2025

La **forma tradicional**: con `require()`. Este es el sistema tradicional de Node.js, usado durante muchos años. Funciona sin necesidad de configurar nada. Hoy en día está en desuso excepto en proyectos más antiguos o de prueba. El uso de `require()` para importar módulos en Node.js, asociado al sistema CommonJS, se considera la forma tradicional o "antigua" y se está desaconsejando a favor de la sintaxis `import/export` del sistema ECMAScript Modules (ESM).

En este curso usaremos la forma moderna a través de la sintaxis `import...from`. Esta es la forma moderna de importar, igual que en JavaScript del navegador (ES6). Permite escribir código más limpio y usar `export` e `import` directamente. Para usar este modelo hay que hacer un cambio en el `package.json` o crearlo si aún no lo tienes. Una vez hecho este cambio la sintaxis `require()` deja de funcionar. Tenlo en cuenta.

```
api > {} package.json > ...  
1   {  
2   "type": "module"  
3   }  
4
```

¿Qué tipos de modules existen en [node.js](https://nodejs.org/)? Tenemos que distinguir tres tipos.

Tipos de módulos en Node.js

Tipo de módulo	Descripción
◆ Nativo	Incluido en Node.js (no necesitas instalar nada). Ej: <code>fs</code> , <code>http</code> , <code>path</code> .
◆ De terceros	Instalados con <code>npm</code> . Ej: <code>express</code> , <code>axios</code> , <code>chalk</code> .
◆ Propio	Archivos <code>.js</code> que tú creas con funciones o clases reutilizables.

Módulos Propios.

Un módulo propio (también llamado módulo local) es un archivo JavaScript creado por ti mismo que contiene código reutilizable (funciones, objetos, clases, variables...) y que puedes importar en otros archivos de tu proyecto.

Vamos a crear un módulo propio.

Vamos a crear un módulo que se va a llamar sumar: una función que recibe dos parámetros y los devuelve. Luego vamos a llamar a ese módulo en otro archivo que lo vamos a llamar [sumando.js](#) y llamaremos a la función.

Aquí tienes el ejemplo.

```
JS sumar.js X JS sumando.js JS promises.js
utils > JS sumar.js > sumar
1   export function sumar(a, b) {
2     return a + b;
3   }
```

Este es nuestro módulo. Ahora lo importamos.

```
JS sumar.js JS sumando.js X JS promises.js
JS sumando.js > ...
1   import { sumar } from './utils/sumar.js';
2   const resultado = sumar(4, 7);
3   console.log('Resultado de la suma:', resultado);
4
```

Ejercicio.

Ahora crea tu un módulo para restar y ejecuta la función pasándole dos números.

Módulos nativos fundamentales.

Vamos a ver los módulos nativos de [Node.js](#). Node.js no solo nos permite ejecutar JavaScript fuera del navegador, sino que también nos ofrece un conjunto de herramientas integradas llamadas módulos nativos.

Estos módulos forman parte del propio núcleo de Node y nos permiten realizar tareas muy comunes sin necesidad de instalar nada adicional con npm.

Podemos imaginar los módulos nativos como las “herramientas básicas” de [Node.js](#), listas para usar en cualquier proyecto: leer archivos, trabajar con rutas, crear servidores, manejar eventos, etc.

Por ejemplo (no són los únicos pero sí los más usados?)

- fs (File System): permite leer, crear y modificar archivos del sistema operativo.
- path: ayuda a gestionar rutas y nombres de archivos de forma segura.
- http: sirve para crear servidores web y manejar peticiones.
- os: nos da información sobre el sistema operativo (CPU, memoria, etc.).
- events: permite crear y escuchar eventos personalizados.
- Crypto. encriptar, crear contraseñas seguras, firmar datos o generar tokens.

fs (File System)

Permite leer, escribir, copiar o eliminar archivos en el sistema operativo. Ideal para guardar logs, manejar plantillas o generar archivos dinámicos.

Inma Contreras@2025

Vamos a crear dos archivos uno que se va a llamar datos.txt con un texto el que queramos y otro que va a leer nuestro archivo usando el módulo nativo fs.

```
JS leerarchivo.js > ...
1  import fs from 'node:fs';
2
3  // Leer el archivo correcto (datos.txt)
4  fs.readFile('datos.txt', 'utf8', (err, datos) => {
5    if (err) {
6      console.error('❌ Error al leer el archivo:', err);
7      return;
8    }
9
10   console.log('📄 Contenido del archivo:\n');
11   console.log(datos);
12 });
13
```

Entendiendo el código. Le dices a Node que quieres usar el módulo nativo fs (File System). Por favor, abre el archivo llamado datos.txt y léelo como texto (utf8). Devuelve (parámetros declarados en la función) una variable que se llama datos o bien devuélveme un error. Si algo sale mal (por ejemplo, el archivo no existe o no tiene permisos), se muestra el error en consola. Si todo fue bien, se muestra el contenido del archivo en pantalla devolverá la variable datos que contiene el texto que había dentro de datos.txt.

Pruébalo en tu Visual Code y analiza con tus palabras.

¿Qué más podemos hacer con el módulo fs?

Leer archivos (fs.readFile)

Escribir nuevos (fs.writeFile)

Copiar, renombrar o borrar archivos (fs.copyFile, fs.unlink, etc.).

Ejercicio.

Intenta crear un archivo; esta vez con fs vamos a crear un archivo nuevo con un texto "Hola soy un archivo nuevo creado desde [Node.js](#)".

Explica cómo se usa el módulo fs

path

Ayuda a trabajar con rutas de archivos y directorios de forma compatible entre sistemas (Windows/Linux). Muy útil al configurar Express o manejar rutas absolutas. El módulo path no se usa directamente en el día a día del desarrollo full stack, pero es fundamental entenderlo, porque Node lo usa constantemente para localizar archivos, carpetas y recursos estáticos.

Vamos a ver un ejemplo de path por ejemplo para buscar un archivo que se va a llamar [path.js](#) y está dentro de una carpeta que se llama api. Creamos un archivo que se va a llamar [buscar.js](#) y lo vamos a crear fuera de la carpeta api.


```

JS buscar.js ×
JS buscar.js > ...
1 // Importamos el módulo nativo 'path'
2 import path from 'node:path';
3
4 // Unimos carpetas y archivo con path.join()
5 // Esto crea una ruta válida en cualquier sistema operativo
6 const rutaArchivo = path.join('api', 'path.txt');
7
8 console.log('✿ Ruta relativa unida:', rutaArchivo);
9
10 // i queremos la ruta ABSOLUTA (completa)
11 const rutaAbsoluta = path.resolve('api', 'path.txt');
12
13 console.log('📌 Ruta absoluta:', rutaAbsoluta);
14

```

Ejercicio.

Tu ejercicio consiste en unir el módulo anterior fs con este nuevo módulo que es tu brújula de archivos. Tendrás que buscar el archivo [path.txt](#) y tienes que escribir en el console.log ("soy un texto nuevo creado con fs"). Crea un nuevo archivo por ejemplo [pathfs.js](#) e intenta que funcione.

Modifica el texto que escribes y vuelve a lanzarlo. ¿Qué ocurre?

Ahora quiero que investigues y pruebes fs.append.

http

El módulo http permite a Node.js crear servidores web que pueden escuchar peticiones del navegador y enviar respuestas.

Gracias a él, tu ordenador puede comportarse como un servidor real:

Cuando alguien entra en una dirección como `http://localhost:3000`, Node recibe la petición y responde.

http es el corazón del backend en Node.js, porque permite la comunicación entre el cliente (frontend) y el servidor (backend).

Ejercicio. ¿Para qué vale el módulo HTTP? Busca los métodos y explica. ¿Qué utilidad piensas que podrían tener?

En una comunicación web intervienen dos partes:



Servidor (Node.js) Espera y responde a las peticiones.



Cliente (navegador) Envía una petición (por ejemplo, al entrar en una URL).

Cuando el cliente envía una petición, Node la recibe como un objeto `req`, y responde con otro objeto llamado `res`.

Ejercicio. Explica qué puertos puedes usar y cuáles son los habituales.

¿Qué es un servidor? Ya hemos visto el concepto servidor y ahora lo vamos a ver funcionando. Un **servidor HTTP** es un programa que escucha peticiones (como cuando visitas una web) y responde con contenido (texto, HTML, JSON...). En Node.js, podemos crear uno desde cero usando el módulo nativo http. Un servidor es el que hemos creado en el ejercicio anterior que abre un puerto pero este servidor responde con texto plano.

Crea un archivo y llámalo [servidor.js](#)

Vamos a crear un pequeño servidor.

```
// Archivo: server.js
import http from 'node:http';

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
  res.end('¡Hola desde Node.js! 🚀 \nTu primer servidor está funcionando.');
```

```
});

server.listen(3000, () => {
  console.log('🌐 Servidor ejecutándose en http://localhost:3000');
  console.log('💡 Presiona Ctrl+C para detener el servidor');
```

```
});
```

Analizamos el código.

El módulo `http` es una de las piezas centrales de Node.js.

Permite **crear servidores web** capaces de recibir peticiones del navegador (o de otro cliente) y responder con datos o páginas. El módulo nativo de Node.js se llama `http` y es lo que permite a tu servidor **escuchar** en un puerto (como el 3000) y **entender** las peticiones HTTP que llegan.

Ejemplo mental:

El **cliente** (el navegador) pide:

“Dame la página principal.”

El **servidor** responde:

“Aquí tienes el contenido.”

Crear el servidor

```
const server = http.createServer((req, res) => {
  // ...
});
```

Inma Contreras@2025

El método `createServer()` recibe una función de callback que se ejecuta cada vez que llega una petición HTTP. Luego veremos que significa “callback” te adelanto que es “no te quedes parado y sigue trabajando”.

Cada vez que el servidor recibe una petición, Node llama automáticamente a tu función y le pasa dos objetos especiales:

<code>req</code> (request)	Representa la petición del cliente	Contiene datos como la URL solicitada, el método (GET, POST, etc.), cabeceras, cuerpo, etc.
<code>res</code> (response)	Representa la respuesta del servidor	Permite definir qué se devuelve al cliente: el código de estado (200, 404...), el tipo de contenido y el propio cuerpo de la respuesta.

```
const server = http.createServer((req, res) => {  
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });  
  res.end('¡Hola desde Node.js! 🚀\nTu primer servidor está funcionando.');
```

});

No son palabras reservadas, son nombres elegidos por convención

(son abreviaturas de request y response).

Tú podrías llamarlos petición y respuesta, aunque casi todo el mundo usa req y res.

Escuchar en un puerto

Para que el servidor funcione, debemos decirle **en qué puerto escuchará** (el canal de comunicación):

```
server.listen(3000, () => {  
  console.log('🌐 Servidor ejecutándose en http://localhost:3000');  
  console.log('💡 Presiona Ctrl+C para detener el servidor');
```

});

Inma Contreras@2025

En este primer ejemplo no hemos usado req no nos importa que nos pida el cliente siempre vamos a responder igual pero req podrían ser rutas por ejemplo /contacto /tienda y entonces si toma valores.

Ejercicio. Aquí tienes parte del código. Te aviso que no está completo. Piensa y mira a ver que puede faltar para que funcione.

```
// 2 Analizamos la ruta que el cliente solicita
if (req.url === '/' || req.url === '/inicio') {
  res.statusCode = 200; // código 200 = OK
  res.end('🏠 Bienvenido a la página principal');
}

// 📞 Ruta de contacto
else if (req.url === '/contacto') {
  res.statusCode = 200;
  res.end('📞 Esta es la página de contacto.\nPuedes escribirnos a contacto@nodeejemplo.com');
}

// 🛒 Ruta de tienda
else if (req.url === '/tienda') {
  res.statusCode = 200;
  res.end('🛒 Bienvenido a la tienda online.\nPronto habrá ofertas y productos nuevos.');
```

```
// ❌ Si la ruta no existe
else {
  res.statusCode = 404; // código 404 = No encontrado
  res.end('❌ Página no encontrada. Prueba con /contacto o /tienda');
}
});
```

🕒 timers

El módulo **timers** en Node.js forma parte del núcleo del lenguaje y **permite programar tareas para que se ejecuten después de un tiempo o de forma repetida**. Incluye las funciones setTimeout, setInterval y setImmediate para manejar tareas asíncronas.

Es decir, nos sirve para **controlar el tiempo de ejecución** de partes del programa sin bloquear el hilo principal.

¿Para qué sirve?

Simular **retrasos** o **esperas** (por ejemplo, al acceder a una base de datos o a una API).

Programar **tareas periódicas** (limpieza de logs, actualización de datos, envío de notificaciones, etc.).

Controlar la **asincronía** en ejemplos de *callback* o *promesas*. Así es como lo usaremos en las siguientes lecciones.

Coordinar el tiempo de respuesta entre un **servidor Node (backend)** y un **cliente HTML/JS (frontend)**.

Sintaxis básica

js

```
setTimeout(función, tiempo_ms);  
setInterval(función, tiempo_ms);  
setImmediate(función);
```

 Donde `tiempo_ms` se mide en **milisegundos** (1000 ms = 1 s).

♦ setTimeout()

Ejecuta una función una sola vez después del tiempo indicado.

```
setTimeout(() => {  
  console.log("🕒 Han pasado 2 segundos");  
}, 2000);
```

♦ setInterval()

Ejecuta una función de forma repetida cada cierto intervalo de tiempo.

```
setInterval(() => {  
  console.log("🔄 Esto se repite cada segundo");  
}, 1000);
```

Para detenerlo, se usa clearInterval(id):

```
const temporizador = setInterval(() => {  
  console.log("⌚ trabajando...");  
}, 1000);
```

```
setTimeout(() => {  
  clearInterval(temporizador);  
  console.log("✅ Temporizador detenido");  
}, 5000);
```

API en Node.js

Para que el front se comuniquen con el back hace falta crear una API. Una API permite que dos programas se comuniquen. La api responderá con los datos que se soliciten. En el desarrollo web, una API es un **servidor que responde con datos** (normalmente en formato JSON) cuando el frontend hace una petición.

API son las siglas de Application Programming Interface, que en español significa Interfaz de Programación de Aplicaciones.

Una API en Node.js es un servidor que:

- Escucha peticiones HTTP (GET, POST, etc.)
- Analiza la URL (/api/saludo, /api/hora)
- Devuelve datos en formato JSON
- Usa códigos de estado HTTP (200, 404, etc.)

Inma Contreras@2025

Todas las apps modernas usan APIs para comunicarse con el backend. Permite crear tu propio servidor de datos. Frameworks como Express.js se basan en este patrón para simplificar el desarrollo.

Crea esta Api

```
import http from 'node:http';

const server = http.createServer((req, res) => {

  // Cabeceras: CORS y tipo de respuesta JSON

  res.setHeader('Access-Control-Allow-Origin', '*');

  res.setHeader('Content-Type', 'application/json; charset=utf-8');

  //// define cabecera write envía cabecera y luego no permite cambios por eso
  no funciona

  // GET /api/saludo

  if (req.url === '/api/saludo' && req.method === 'GET') {

    const data = { ok: true, mensaje: 'Hola desde Node API 🙌', modulo:
'UF1844' };

    res.writeHead(200);

    return res.end(JSON.stringify(data)); // objeto JS → texto JSON
  }

  // GET /api/hora

  if (req.url === '/api/hora' && req.method === 'GET') {

    const ahora = new Date();

    const hh = String(ahora.getHours()).padStart(2, '0');
```



```
const mm = String(ahora.getMinutes()).padStart(2, '0');

res.writeHead(200);

return res.end(JSON.stringify({ ok: true, hora: ` ${hh}:${mm}` }));

}

// Cualquier otra ruta

res.writeHead(404);

res.end(JSON.stringify({ ok: false, error: 'No encontrado' }));

});

// Puerto 3000

server.listen(3000, () => {

  console.log('✅ API en http://localhost:3000 (GET /api/saludo, /api/hora)');

});

// 🕒 Cerrar automáticamente después de 5 segundos

setTimeout(() => {

  server.close(() => console.log("🛑 Servidor cerrado automáticamente")); // lo usamos sólo en entornos de pruebas

}, 5000);
```

¿Qué acabamos de crear?

En este ejemplo se construye una API utilizando sólo el módulo nativo http de Node.js. Esto significa que no se usa ningún framework externo como Express: todo el flujo de la petición y la respuesta se controla directamente con las herramientas que Node proporciona de forma predeterminada.

El objeto JSON.

JSON (JavaScript Object Notation) es un formato de texto que se usa para intercambiar datos entre programas, servidores o aplicaciones web.

Su nombre viene de “notación de objetos de JavaScript”, porque su sintaxis se parece a los objetos de JS, aunque en realidad JSON es solo texto.

```
const usuarios = [  
  
  {  
  
    nombre: "Inma",  
  
    edad: 25,  
  
    curso: "Node.js"  
  
  },  
  
  {  
  
    nombre: "Sofía",  
  
    edad: 22,  
  
    curso: "React.js"  
  
  },  
  
];
```

Inma Contreras@2025

Se hace una declaracion de variables y se abren corchetes. Se declara texto plano con el concepto de claves : valores con el matiz de que ambos van entre comillas. Digamos que es texto plano ordenado.

JSON es **ligero, universal y legible**. Tanto el navegador (frontend) como Node.js (backend) pueden entenderlo sin convertirlo a otro formato.

En JavaScript (y por tanto en Node.js) existe un objeto global llamado **JSON** con dos métodos muy importantes:

2 Los dos métodos principales del objeto **JSON**

Método	Qué hace	Tipo de valor que devuelve
<code>JSON.stringify(obj)</code>	Convierte un objeto JavaScript → a texto JSON	 <i>string</i>
<code>JSON.parse(texto)</code>	Convierte un texto JSON → a objeto JavaScript	 <i>objeto</i>

Cuando envías una **respuesta desde el servidor**, el navegador solo puede recibir **texto**. Así que el flujo es:

- 1 Creas un objeto con los datos
- 2 Lo conviertes a texto con `JSON.stringify()`
- 3 Lo envías con `res.end()`

Ejemplo.

```
import http from 'node:http';

const server = http.createServer((req, res) => {

  res.setHeader('Content-Type', 'application/json; charset=utf-8');

  const datos = { mensaje: '👋 Hola desde Node.js', ok: true };

  // Convertimos a texto JSON antes de enviar
```

```
res.end(JSON.stringify(datos));});
```

```
server.listen(3000);
```

4 Cómo se usa con `req` (para leer datos entrantes)

Cuando el cliente envía un `POST` con JSON, el servidor lo recibe como texto.

Así que el flujo inverso es:

- 1 Node recibe el texto → `req.on('data')`
- 2 Lo concatenas en una variable
- 3 Lo conviertes a objeto con `JSON.parse()`

```
let body = '';
req.on('data', chunk => {
  body += chunk; // acumulamos trozos de datos
});
req.on('end', () => {
  const datos = JSON.parse(body); // texto → objeto
});
```

```
OBJETO JavaScript → JSON.stringify() → TEXTO JSON → (viaja por HTTP)
                        ↓
                        JSON.parse() ← TEXTO JSON ← (recibido del servidor)
```

```
import http from "http";

const server = http.createServer((req, res) => {

  if (req.method === "POST") {

    let cuerpo = "";

    // 📦 Los datos llegan en fragmentos

    req.on("data", (chunk) => {

      cuerpo += chunk.toString(); // convierte el buffer a texto

    });

    // 📁 Cuando termina de llegar todo:

    req.on("end", () => {

      // 🔍 Aquí usamos JSON.parse()

      const datos = JSON.parse(cuerpo);

      console.log(datos.nombre); // → "Inma"

      console.log(datos.curso); // → "Node.js"

      res.end("Datos recibidos correctamente");

    });

  }

});

server.listen(3000, () => console.log("Servidor en puerto 3000"));
```

Aquí explicamos el cuando se aplica parse, parse es el método de JSON que recibe un JSON y lo convierte a objeto. Por eso, en este punto interviene el método Post que envía datos y crea sentencias desde el navegador hasta el

servidor. Parse es lo contrario a Stringify y a la vez complementarios. Strngify funciona en el envío para poner los datos JSON en la “carretera” y parse funciona en el destino para convertir los datos recibidos en datos operables (objetos).

¿Y que es shunk?

El objeto req en Node.js no es un simple objeto, es un stream (flujo de datos legible, o Readable Stream). Es decir: los datos no llegan todos de golpe, sino en trozos (chunks), sobre todo cuando el cuerpo de la petición (body) es grande o viene por red.

on() es un método de los streams de [Node.js](#).

Este método tiene esta sintaxis

```
req.on(evento, callback);
```

Donde:

- "data" → es el tipo de evento al que quieres reaccionar (se dispara cada vez que llega un trozo de datos). 'data' es una etiqueta de texto que Node usa internamente para saber: “cuando llegue un trozo de datos, ejecuta esta función”.
- callback → es la función que se ejecuta cuando ocurre ese evento.
- “chunk” es el trozo que voy a recibir. El nombre chunk se usa por convención (significa pedazo o trozo), pero no es una palabra reservada, sólo una variable local que tú defines.

**8 En resumen**

Elemento	Qué hace
<code>req</code>	Es un stream legible (flujo de datos entrante)
<code>req.on('data', callback)</code>	Se ejecuta cada vez que llega un trozo de datos
<code>chunk</code>	Es ese trozo de datos (un <code>Buffer</code>)
<code>req.on('end', callback)</code>	Se ejecuta cuando ya se recibieron todos los trozos
<code>body += chunk</code>	Vas acumulando los trozos en una variable

Matar las conexiones.

Primero prueba con Ctrl +C que detiene el proceso actual si te pregunta algo le dices que Y.

Pero hay veces en entornos de pruebas que abrimos y cerramos el mismo puerto muchas veces en que el puerto se puede quedar en estado “zombie” entonces usa estos estos comandos en tu terminal (estamos hablando de Windows).

Busca qué proceso usa el puerto (por ejemplo el 3000 o 5000):

netstat -ano | findstr :3000

Te mostrará una línea con un número final, el PID (identificador del proceso).

Mata el proceso con ese PID:

taskkill /PID 12345 /F

Sustituye 12345 por el número que te haya salido.

Esto libera el puerto inmediatamente.

RECORDATORIO TÉCNICO.

¿Qué es un objeto en JavaScript?

Un objeto es una estructura que agrupa datos y comportamientos relacionados bajo un mismo nombre. Podemos pensar en él como una “caja” con pares clave-valor, donde cada clave (propiedad) tiene un valor asociado (que puede ser un dato o una función).

```
const usuario = {  
  nombre: "Ana",  
  edad: 35,  
  curso: "Node.js",  
};
```

¿Por qué son tan importantes?

Porque JavaScript está construido sobre objetos.

Casi todo lo que usamos es un objeto o se comporta como tal:

- Los arrays ([]) son objetos especiales que almacenan listas.
- Las funciones también son objetos (pueden tener propiedades).
- El DOM del navegador (document, window, Element, etc.) está formado por objetos.
- En Node.js, el propio req, res o los módulos son objetos.
- Cuando usamos fetch(), la respuesta es un objeto Response.
- Incluso los errores (Error) o las fechas (Date) son objetos.

 En resumen:

En JavaScript, los objetos son la forma natural de representar y manejar la información.

Ejecuta este código para ver como viajan los datos del servidor al navegador.

// Servidor HTTP en Node


```
import http from "node:http";

const usuarios = [
  { nombre: "Ana", edad: 35 },
  { nombre: "Luis", edad: 28 },
  { nombre: "Inma", edad: 32 }
];

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "application/json; charset=utf-8");
  res.end(JSON.stringify(usuarios)); // 👉 enviamos el array como JSON
});

server.listen(3000, () => console.log("🚀 Servidor en http://localhost:3000"));
```

Cuando lleguen al navegador éste lo tendrá que volver a convertir (parse) a un objeto para trabajar sobre él. El navegador usa objetos para mostrar y manipular datos en la interfaz. El servidor usa objetos para procesar, almacenar y enviar información. El formato JSON es el “puente” que traduce los objetos a texto para comunicarse entre ambos.

Programación Asíncrona (El motor de Node)

Node.js funciona sobre el motor **V8 de Google** y se basa en un sistema de ejecución llamado **event loop** (*bucle de eventos*).

Inma Contreras@2025

Gracias a este modelo, Node puede manejar **operaciones asíncronas** sin bloquear el hilo principal.

- Lectura/escritura de archivos (fs)
- Peticiones HTTP (http, fetch)
- Acceso a bases de datos
- Timers (setTimeout, etc.)

Esto significa que Node no se queda “esperando” a que termine una tarea lenta: le pide al sistema operativo que la ejecute en segundo plano y sigue con el resto del código.

Cuando la tarea finaliza, el event loop recibe una señal y ejecuta la función asociada al resultado.

En muchos lenguajes por ejemplo C++ o PHP, cuando ejecutas una función, el programa espera a que termine antes de seguir (modo síncrono).

En Node.js, muchas operaciones (leer un archivo, hacer una petición HTTP, acceder a la base de datos...) se hacen de forma **asíncrona**:

- Pasa la tarea al sistema y **sigue ejecutando** el resto del código.
- Cuando la tarea termina, **avisa mediante un callback o una promesa**.

Imagina que pones la lavadora. No te quedas mirando la lavadora hasta que termine; pones la lavadora y mientras tanto haces otras tareas.

Cuando la lavadora pita (callback), recoges la ropa (respuesta lista). El **event loop** es el “cerebro” que coordina las tareas.

Node tiene un único hilo principal (single thread), pero puede delegar tareas a otros hilos o al sistema operativo. Cuando esas tareas terminan, el event loop las recoge y ejecuta su callback.

Concepto de Node.js/Event Loop	Analogía de la Lavadora	Explicación
Hilo Principal (Single Thread)	Tú (la persona que lava) 🧑	Eres el único que puede tomar decisiones (ejecutar código JavaScript). Solo puedes hacer una cosa a la vez: cargar la lavadora, doblar la ropa, o descargar la ropa limpia. No puedes hacer dos de estas a la vez.
Delegar Tareas (Asincronía)	Poner la Lavadora 🧺	Una tarea que lleva mucho tiempo (Input/Output, como una solicitud a una base de datos o a la red). Una vez que la pones a funcionar, no tienes que esperar a su lado.
Otros Hilos/Sistema Operativo	La Lavadora en Sí ⚙️	Es el mecanismo que hace el trabajo pesado en segundo plano (lavar la ropa). Mientras la lavadora trabaja, tú estás libre para hacer otras cosas.
El Event Loop (El Cerebro)	El Temporizador y el Oído 🕒	Es el que coordina la situación. Constantemente revisa si la lavadora ha terminado. No está haciendo la acción de lavar, solo está pendiente de si el trabajo delegado terminó.
El Callback	La Ropa Limpia (la señal de fin) ✅	Es la acción final que debes hacer tú (el Hilo Principal) una vez que la tarea delegada termina. Por ejemplo, la función que dice: "Ahora que la lavadora pitó, voy a sacar y tender la ropa (ejecutar el callback)".

Ahora vamos a ver como [Node.js](#) consigue hacer esto, es decir cómo consigue esperar sin bloquearse. Event loop es un concepto genérico, callback han quedado "obsoletas" desde la versión **ES6 (ECMAScript 6)** la **sexta versión del estándar de JavaScript**, publicada oficialmente en **2015**. ES6 modernizó el lenguaje completamente, haciéndolo más potente, legible y orientado a proyectos grandes, tanto en el **navegador** como en [Node.js](#). Esta versión introduce en Js las template literals, las funciones flechas, la declaración con let, la posibilidad de importar módulos sin usar require y las promesas.

Para entender cómo consigue "esperar sin bloquearse", debemos conocer cuatro conceptos fundamentales:

1. Event Loop – el organizador de tareas.
2. Callbacks – la forma tradicional de manejar tareas que tardan.
3. Promesas (Promises) – la evolución moderna de los callbacks.

4. Async / Await – la forma más legible y actual de trabajar con promesas.

¿Qué es el Event Loop?

El event loop es el organizador de tareas de Node.js. Node tiene una sola línea de trabajo principal (un hilo), pero cuando encuentra una tarea lenta, la manda a otro sitio (por ejemplo al sistema operativo o a un hilo interno) y sigue con el resto del código.

Cuando esa tarea termina, el event loop la devuelve a la cola de tareas y la ejecuta cuando haya un hueco libre.

Los callback

Una **callback** es una función que se pasa como argumento a otra función para que se ejecute **más tarde**, cuando esa otra función haya terminado una tarea. La palabra *callback* significa literalmente “**llamar de vuelta**”. No es una palabra reservada de JavaScript: podrías llamarla como quieras, pero se usa ese nombre cuando la función se ejecuta después de otra.

Sintaxis de callback

Ejemplo sin callback (funciones normales)

En este caso, las funciones se ejecutan **de forma síncrona**, una detrás de otra:

```
function saludar() {  
  console.log("👋 Hola");  
}
```

```
function despedir() {
```

Inma Contreras@2025

```
    console.log("👋 Adiós");  
}
```

```
saludar();  
despedir();
```

Salida por consola:

👋 Hola

👋 Adiós

Ejemplo con callback (función que se ejecuta más tarde)

Ahora la función saludarDespues() usa un retraso con setTimeout() para simular una tarea que tarda (por ejemplo, una lectura de datos o conexión a una API). Cuando termina, llama a la función que pasamos como callback.

```
function saludarDespues(callback) {  
    // Simulamos una tarea que tarda 2 segundos (por ejemplo, leer datos o  
    conectarse a una API)  
    setTimeout(() => {  
        callback(); // 👉 ejecuta la función pasada como argumento  
    }, 2000);  
}  
  
// Pasando una función anónima flecha como callback  
saludarDespues(() => console.log("👋 Soy una función que me ejecuto después  
de dos segundos"));
```

```
// 🌿 Ejemplo 2: usando una función con nombre (más legible)

function mostrarMensaje() {

  console.log("👋 Soy la función callback de una función no anónima");

}

saludarDespues(mostrarMensaje);

// Estos mensajes se muestran primero porque no tienen retardo

console.log(" Yo voy antes, no tengo retraso");

console.log(" Yo también me adelanto");
```

Así funcionan muchas operaciones asíncronas en JavaScript: el programa no se detiene, y el callback se ejecuta cuando termina la tarea pendiente.

⚠ El "Callback Hell"

Cuando varias tareas dependen unas de otras, los callbacks pueden quedar **anidados** dentro de otros, formando un código difícil de leer y mantener:

```
tarea1(() => {
  tarea2(() => {
    tarea3(() => {
      console.log("✅ Todo hecho");
    });
  });
});
```

Este patrón se conoce como “**Callback Hell**” (el infierno de los callbacks).

Las Promesas

Para evitar ese problema, JavaScript introdujo las Promesas, una forma mucho más limpia y moderna de manejar la asincronía. Las promesas permiten encadenar tareas sin anidarlas y manejar los errores de forma más clara con `.then()`, `.catch()` o la sintaxis `async/await`.

💡 A partir de aquí, trabajaremos con Promesas, que son el estándar actual en Node.js y en todo JavaScript moderno.

Las promesas no eliminan los callbacks, pero sí los sustituyen como forma moderna y más limpia de manejarlos.

Una promesa en Node.js, se puede manipular mediante la palabra reservada **promise** y se encuentra formada principalmente por retornar dos valores de respuesta: **resolve()** que devuelve el valor cuando es exitosa la operación realizada de la tarea asíncrona y **reject()** cuando la operación arroja algún error. Resolve y reject no son palabras reservadas son dos parámetros que por convención los llamamos así pero funciona con cualquier nombre. Esta es la sintaxis básica de una promesa.

```
const miPromesa = new Promise((resolve, reject) => {  
  // 🌱 Aquí dentro se escribe el código que puede tardar o fallar  
  // Si todo va bien:  
  resolve(valor);  
  
  // Si ocurre un error:  
  // reject(error);  
});
```

Es decir una promesa devolverá cuando tenga el resultado un valor de éxito (resolve) o un valor de fracaso (reject).

```
const miPromesa = new Promise((resolve, reject) => {  
  const exito = true; // cambiamos a false para probar el error  
  if (exito) {  
    resolve("✅ Operación completada con éxito");  
  } else {  
    reject("❌ Ha ocurrido un error");  
  }  
});
```

Ahora mismo creamos la promesa pero hay que consumirla y se consume con then y catch. Then llama al éxito. Catch llama al fracaso. Then y catch son métodos de promise que es un objeto no es una función por eso decimos consumirla y no llamarla.

Lo importante es el orden de los parámetros de la promise dos parámetros en su orden exacto: primero el éxito que lo encuentra el método then y luego el fracaso que lo encuentra catch.

```
//aquí estamos consumiendo la promesa  
  
miPromesa  
  .then((resultado) => console.log(resultado)) // si todo sale bien  
  .catch((error) => console.error(error)); // si algo falla
```

En resumen.

Definimos los parámetros **resolve y reject**. Resolve y reject son dos funciones que recibe la promesa automáticamente. New Promise() no ejecuta el código de inmediato, sino que devuelve un objeto con dos posibles resultados futuros: resolve (éxito) o reject (error).

El valor que obtienes (miPromesa) es un objeto Promise, y todos los objetos de tipo Promise ya traen incorporados estos tres métodos: .then (), catch () y finally().

Los métodos .then(), .catch() y .finally() son métodos nativos del objeto Promise en JavaScript.

El método .then() se ejecuta cuando la promesa se cumple correctamente, es decir, cuando se llama a resolve() y el proceso finaliza con éxito.

Por su parte, el método .catch() se ejecuta cuando la promesa se rechaza, lo que ocurre al invocar reject() o si se produce un error durante la ejecución.

Finalmente, el método .finally() se ejecuta siempre, tanto si la promesa se cumple como si falla, y se utiliza para realizar acciones de cierre o limpieza una vez que el proceso ha terminado, independientemente del resultado.

Si queremos usar la promesa la tenemos que consumir (consumir porque es un objeto pero es como llamar si fuera una función)

```
//aquí estamos consumiendo la promesa fíjate bien que no se cierran las sentencias de la promesa
miPromesa
  .then((resultado) => console.log(resultado)) // si todo sale bien
  .catch((error) => console.error(error)) // si algo falla
  .finally(() => console.log("🏁 Soy el final, pase lo que pase"))://aunque haya error o éxito finally se ejecuta
```

```
//cambia éxito a false para ver el manejo del error
```

Cuando resolve o reject se ejecutan la promesa cambia de estado

🚦 Estados de una Promesa

Estado	Descripción
Pending	La promesa se ha creado pero aún no ha terminado.
Fulfilled	La operación se completó correctamente (<code>resolve</code>).
Rejected	La operación falló (<code>reject</code> o un error lanzado).

Cómo puedes imaginar en proyectos reales, el valor de éxito no se define a mano, sino que lo determina el frontend o el propio flujo de la aplicación (por ejemplo, si el usuario inició sesión correctamente, si se guardaron los datos, etc.).

De este modo, el backend decide resolver o rechazar la promesa según la información recibida del cliente.

Métodos para manejar Promesas

Todas las promesas tienen tres métodos nativos:

Método	Cuándo se ejecuta
<code>.then()</code>	Cuando la promesa se cumple (resolve).
<code>.catch()</code>	Cuando la promesa se rechaza (reject o error).
<code>.finally()</code>	Siempre, tanto si hay éxito como error.

La aportación de ES8: Async y await.

Después de ES6 (2015, cuando se introdujeron las Promesas), en ES8 (2017) JavaScript añadió las palabras clave **async** y **await** para hacer el código asíncrono más fácil de leer y escribir, evitando las cadenas largas de `.then()` y `.catch()`. `async` y `await` son palabras reservadas del lenguaje JavaScript creadas para trabajar con promesas.

ASYNC

La palabra clave **async** se coloca delante de una función para indicarle a JavaScript que esa función devolverá un objeto promesa es decir no se ejecuta su código de forma inmediata y síncrona, si no que es una promesa una tarea que puede tardar y cuyo resultado llega en el futuro.

Inma Contreras@2025

```
// ⚡ Función async que espera con await

async function ejecutarEsperando() {

  console.log("🚀 Yo ya me he ejecutado, no tengo await");

  const resultado = await tareaConRetardo(); // 🕒 aquí espera

  console.log(resultado);

  console.log("⚡ Soy la segunda tarea y espero a la primera");

  console.log("✅ Fin del proceso con espera");

}

// 🚀 Ejecutamos

ejecutarEsperando();
```

AWAIT

La palabra clave **await** sólo se podía usar dentro de una función marcada como `async`, y sirve para esperar a que una promesa se resuelva antes de continuar con el código. Hoy en día desde la versión ES13 del 2022 también puede ir sólo pero con una condición que tu proyecto esté usando `type:module` (import)

async + await dentro de función (sirve en cualquier entorno);

Y este desarrollo es el que vamos a ver ahora: el uso de ambos términos unidos.

Con `await`, el código se pausa dentro de la función hasta que la promesa termina, pero sin bloquear el resto del programa.

Prueba este código.

```
// Función que devuelve una promesa (simula tarea con retardo)

function tareaConRetardo() {

  return new Promise((resolve) => {

    console.log("🕒 Iniciando tarea con retardo...");
```

Inma Contreras@2025

```

    setTimeout(() => {
        resolve("🎯 Tarea completada tras 2 segundos");
    }, 2000);
    });
}

// ⚡ Función async que espera con await

async function ejecutarEsperando() {
    console.log("🚀 Yo ya me he ejecutado, no tengo await");

    const resultado = await tareaConRetardo(); // 👉 aquí espera
    console.log(resultado);

    console.log("⚡ Soy la segunda tarea y espero a la primera");
    console.log("✅ Fin del proceso con espera");
}

// 🚀 Ejecutamos
ejecutarEsperando();

```

Cuando usas `await`, ya no necesitas `.then()` ni `.catch()`, porque el código espera directamente a que la promesa se resuelva o falle.

El propio `await` consume la promesa por ti.

¿Pero y si falla? Ya no hay `reject` ¿cómo controlamos los errores con `await`? aquí entra en juego un bloque: `try...catch`. El **bloque `try...catch`** sirve para atrapar errores (excepciones) que ocurren durante la ejecución del código. Es importante atrapar el error porque si un error no se atrapa subirá hasta el siguiente nivel y seguirá subiendo hasta que encuentre un `catch`. Es lo que se

llama la propagación de errores. Capturar el error es importante para evitar que el programa se rompa y puedas reaccionar de forma controlada. Lo verás también en las conexiones a bases de datos, en las llamadas de fetch e incluso en las conversiones de datos json.

Bloque try...catch

- try { ... }: el código “normal” que puede fallar.
- catch (error) { ... }: se ejecuta solo si algo falla (la promesa se rechazó o se lanzó una excepción).
- finally { ... }: (opcional) se ejecuta siempre, con éxito o error (cerrar recursos, dejar un log, ocultar spinners, etc.).

AWAIT (SIN ASYNC)

Y ahora la cuarta forma de crear y consumir una promesa: **await (sin async)**

```
const promesa = new Promise((resolve, reject) => {  
  const exito = Math.random() > 0.5;  
  setTimeout(() => {  
    if (exito) resolve("✅ Todo salió bien");  
    else reject("❌ Algo falló");  
  }, 1000);  
});
```

```
// ⚙ Manejo con try...catch (por si falla)
```

```
try {  
  const resultado = await promesa;  
  console.log(resultado);  
} catch (error) {  
  console.error("🔥 Error capturado:", error);  
} finally {  
  console.log("🏠 Proceso terminado (éxito o error)");  
}
```

En este código creamos una promesa manualmente con `new Promise(...)`.

- `resolve(...)` → se ejecuta si todo sale bien.
- `reject(...)` → se ejecuta si hay un error.

El `setTimeout()` solo simula que algo tarda (por ejemplo, conectarse a una API o una base de datos).

Así que después de 1 segundo, la promesa se resuelve o se rechaza al azar.

Cómo puedes ver en la creación de promesa hemos vuelto a la `new promise`

La promesa en sí no cambia.

Sigue siendo el mismo objeto que tiene dos posibles resultados:

- `resolve()` → cuando todo sale bien
- `reject()` → cuando algo falla

Lo que **sí cambia entre versiones** es **la forma de consumirla**

Es decir, **cómo esperamos su resultado y cómo manejamos el éxito o el error.**

Conclusión.

Cuando un programa necesita hacer algo que tarda (por ejemplo, esperar 2 s), queremos esperar el resultado sin bloquear el resto del programa.

Ejemplo base

Queremos que nuestro código:

1. Espere 2 segundos
2. Luego muestre un mensaje: "⌚ Tarea completada"

2 Versión 1 — Callback clásico (antes de ES6)

```
function tarea(callback) {  
  setTimeout(() => {  
    callback("⌚ Tarea completada");  
  }, 2000);  
}  
  
tarea(resultado => console.log(resultado));
```

♦ Cómo funciona:

- Pasamos una función (callback) que se ejecutará más tarde.
- El código no espera; cuando setTimeout termina, llama al callback.

♦ Desventajas:

Si hay que hacer varias tareas en cadena, el código se llena de callbacks anidados (callback hell).

⚙️ 3 Versión 2 — Promesas con .then() (ES6 – 2015)

```
function tarea() {  
  return new Promise(resolve => {  
    setTimeout(() => resolve("⌚ Tarea completada"), 2000);  
  });  
}
```

```
tarea()  
  .then(resultado => console.log(resultado))  
  .catch(error => console.error(error));
```

- ♦ Qué mejora:

Ya no pasamos una función, sino que tarea devuelve una promesa.

El código se puede encadenar con .then() y manejar errores con .catch().

- ♦ Aún un poco técnico, con muchos paréntesis y funciones anidadas.

⚙️ 4 Versión 3 — Async / Await dentro de una función async (ES8 – 2017)

```
function tarea() {  
  return new Promise(resolve => {  
    setTimeout(() => resolve("⌚ Tarea completada"), 2000);  
  });  
}
```

```
async function ejecutar() {  
  const resultado = await tarea(); // espera la promesa
```



```
console.log(resultado);  
}
```

```
ejecutar();
```

- ◆ Qué mejora:

Parece código secuencial, más fácil de leer.

async marca la función como asíncrona y permite usar await dentro.

- ◆ Limitación:

Solo puedes usar await dentro de una función async.

Fuera de ella (a nivel global), JavaScript daba error.

⚙️ 5 Versión 4 — Top-Level Await (await “solo”) (ES2022 / Node 14.8+)

```
// package.json → { "type": "module" }
```

```
const promesa = new Promise(resolve => setTimeout(() => resolve("⌚ Tarea  
completada"), 2000));
```

```
const resultado = await promesa; // ✅ await directamente al nivel superior  
console.log(resultado);
```

- ◆ Qué aporta:

Ahora ya no necesitas una función async solo para poder usar await.

Puedes escribir await directamente al nivel superior de tu archivo (si es un módulo ES, .mjs o "type":"module").

- ◆ Ventajas prácticas:

- Ideal para scripts de arranque de servidores (por ejemplo, esperar a la conexión de la base de datos)

- El código es más lineal y limpio, sobre todo en inicializaciones.

6 Analogía de las cuatro versiones

Versión	Cómo lo hace	Analogía de “esperar la lavadora”
Callback	Te avisan cuando termina (callback())	Le dices a alguien: “cuando pite la lavadora, avísame”.
Promise + .then()	Te devuelven un ticket (promesa) y luego lees el resultado con .then()	Te dan un ticket que dice: “te avisaré cuando acabe” y tú miras el ticket más tarde.
Async / Await	Pausas tu función hasta que la lavadora termina	Esperas dentro de una habitación (función async) hasta que pita.
Top-Level Await	Esperas directamente, sin entrar en ninguna función	Estás en casa y puedes esperar a la lavadora sin meterte en ninguna habitación específica.

Versión	Año / ES	Palabras clave	Dónde puedes usarla	Ventajas principales
Callback	Pre-ES6	Ninguna	En cualquier JS	Base de la asincronía
Promise + then()	ES6 (2015)	new Promise, .then()	En cualquier JS	Evita callback hell
Async / Await	ES8 (2017)	async, await	Dentro de funciones async	Código más legible
Top-Level Await	ES2022 (Node 14.8+)	await	Al nivel superior de módulos ES	No necesitas función async

Ejercicio: Simular el estado de un pedido online (pedido.js)

Objetivo

Simular el seguimiento de un pedido que puede llegar correctamente o fallar, usando promesas en JavaScript.

El ejercicio debe mostrar si el pedido fue entregado (resolve) o si hubo un problema (reject).

Instrucciones

- Crea un archivo llamado pedido.js.

- Declara una variable:

```
const entregado = Math.random() > 0.5; // 50% de éxito o error
```

- Crea una promesa que se resuelva si el pedido se entrega correctamente, o se rechace si hubo un error.
- Muestra el resultado por consola usando .then(), .catch() y .finally().
- (Opcional) Haz el mismo ejemplo usando async y await.
- ¿Te atreves a hacerlo sólo con wait?

New Promises	Promesa con Async	Promesa con Async y wait	Promesa solo con wait
<pre>function verificarUsuario() { return new Promise((resolve, reject) => { const ok = Math.random() > 0.5; if (ok) { resolve("✅ Usuario verificado"); // igual que return } else { reject(new Error("❌ Error: usuario no autorizado")); // igual que throw } }); } // Consumimos la promesa verificarUsuario() .then((msg) => console.log(msg)) // captura resolve .catch((err) => console.error(err.message)) // captura reject .finally(() => console.log("🏁 Verificación finalizada"));</pre>	<pre>// ✅ Versión con async async function verificarUsuario() { const ok = Math.random() > 0.5; if (ok) { return "✅ Usuario verificado"; } // resolve implícito else { throw new Error("❌ Error: usuario no autorizado"); // reject implícito } } // Consumimos la promesa verificarUsuario() .then((msg) => console.log(msg)) // captura resolve .catch((err) => console.error(err.message)) // captura reject .finally(() => console.log("🏁 Verificación finalizada"));</pre>	<pre>// ⚡ Versión con await y try/catch async function ejecutarVerificacion() { try { const resultado = await verificarUsuario(); // espera a que se resuelva console.log(resultado); } catch (error) { console.error(error.message); } // captura el reject finally { console.log("🏁 Verificación finalizada (await)"); } } // Ejecutamos ejecutarVerificacion();</pre>	<pre>// Promesa normal function verificarUsuario() { return new Promise((resolve, reject) => { const ok = Math.random() > 0.5; if (ok) { resolve("✅ Usuario verificado"); } else { reject(new Error("❌ Usuario no autorizado")); } }); } // ⚡ Top-level await (sin async) try { console.log("🕒 Verificando usuario..."); const resultado = await verificarUsuario(); // await directamente en el nivel superior console.log(resultado); } catch (error) { console.error(error.message); } finally { console.log("🏁 Verificación finalizada (top-level await)"); }</pre>

Fetch(): Comunicación entre el Frontend y el Servidor

¿Qué es fetch()?

fetch() es una función nativa de JavaScript (no necesitas instalar nada) que permite al **navegador** conectarse con un servidor y pedir o enviar información.

fetch() es el puente que conecta tu frontend (HTML, JS, React...) con tu backend (Node, Express, APIs, bases de datos...).

Se usa para realizar peticiones HTTP , y recibir la respuesta del servidor.

- Es una función del navegador para hacer peticiones HTTP (GET, POST...).
- Devuelve una promesa con un objeto Response.
- Para leer los datos JSON: await res.json().

La función fetch() se utiliza para pedir (GET) y enviar (POST) datos entre el frontend y el backend, y es fundamental en las aplicaciones web modernas.

¿Qué hace exactamente?

1. Envía una petición HTTP a una dirección o URL (por ejemplo, a una API o a tu servidor Express).
2. Espera una respuesta del servidor, que puede contener datos, mensajes o errores.
3. Devuelve una promesa, que se resuelve con un objeto especial llamado Response. Cada vez que haces fetch(), el navegador crea un objeto Response, que representa toda la respuesta HTTP que vino del servidor. El objeto Response tiene propiedades y métodos que permiten inspeccionar y procesar la respuesta.

Cuando haces `fetch()`, el navegador devuelve un objeto llamado **Response**.

Este objeto **no es el dato final**, sino un paquete con toda la información de la respuesta HTTP.

Dentro de él viene el cuerpo (body), que todavía hay que **abrir** con `.json()` o `.text()` para poder usar los datos.

En este punto empezamos a trabajar en el front y en el back. Aquí comenzamos a ser full stack.

```
fetch('https://jsonplaceholder.typicode.com/users')
  .then(res => res.json())    // Transformar texto JSON → objeto JS
  .then(data => console.log(data)) // Mostrar los datos en consola
  .catch(err => console.error("❌ Error en la conexión:", err));
```

¿Qué hace este código?

- `fetch(url)` → realiza la petición a una api externa que te trae datos de usuarios ficticios. El navegador **envía una petición HTTP GET** a esa URL. Por defecto HTTP envía un get y en este caso es lo que queremos pero también puede enviar un POST, un PUT o un DELETE según nuestras necesidades.
- El servidor responde con un **texto JSON** un array de usuarios ficticios porque así está configurada esta api de place holder user (la url que está entre parentesis). Recuerda que `fetch` es el lado del front ahora estamos en un html. Exactamente estamos dentro de un script que se ejecuta en el front.
- `.then(res => res.json())` → convierte la respuesta (texto plano) en un objeto JS. Pero `fetch()` no entrega ese texto directamente. Devuelve una promesa que se cumple con un objeto especial: **Response**. `res` tampoco es una palabra reservada puedes llamarla como quieras pero acostúmbrate a `res` y a `req`.

Ese `res` (de response) es el objeto `Response`. Dentro de él hay un flujo de datos (body) que todavía no está leído.

Cuando hacemos: `res.json()`

estamos diciéndole: "Lee el cuerpo de la respuesta y conviértelo del texto JSON a objeto JS". Y `.json()` también devuelve otra promesa.

Esa promesa se resolverá cuando el navegador haya terminado de leer y convertir los datos. Entonces este fragmento:

```
.then(res => res.json())
```

devuelve una nueva promesa que contiene los datos ya convertidos a objeto.

Y esa promesa pasa su resultado al siguiente `.then()`:

- `.then(data => ...)` → usa esos datos (mostrar, pintar en pantalla, etc.).
- `.catch(err => ...)` → atrapa errores de red o del servidor.

`data` no la defines tú manualmente. Se crea como parámetro de la función del `.then()`, y su valor es el resultado de la promesa anterior (`res.json()`) le puedes dar el nombre que quieras.

En este ejemplo `data` es un array de objetos JavaScript (porque el JSON de esa URL es una lista de usuarios).

Podrías usarlo así:

```
.then(data => {  
  console.log(data);    // Muestra el array completo (mejor no lo uses)  
  console.log(data[0]); // Primer usuario  
  console.log(data[0].name); // Nombre del primer usuario  
})
```




5 Diagrama del flujo de datos

SCSS

```

fetch(url)
  ↓
Promesa → Response
  ↓
res.json() → Promesa → Objeto JS (data)
  ↓
.then(data => { ... }) ← Aquí llega el contenido
  ↓
.catch(err => ...)      ← Si falla algo

```

Recuerda que la conexión entre back y front no siempre es lo inmediata que nos gustaría por eso siempre para garantizar que esa conexión es exitosa usaremos dos conceptos que ya hemos visto:

- Async...await
- Bloque try...catch

```

async function obtenerUsuarios() {
  try {
    const res = await fetch('https://jsonplaceholder.typicode.com/users');
    const datos = await res.json();
    console.log(datos);
  } catch (error) {
    console.error("❌ Error al obtener datos:", error);
  }
}

```

Inma Contreras@2025

```
}
```

```
obtenerUsuarios();
```

Ejercicios.

Ahora empezamos con los ejercicios que nos van a valer de ejemplos.

Lo primero y fundamental va a ser crear una estructura a la que nos tendremos que acostumbrar.

Estructura en proyectos de back y front.

Creamos una nueva carpeta y en ella dos carpetas más: una para el back y otra para el front.

Nomenclatura. Carpeta o archivo backend → todo en minúsculas y, si son varias palabras, separadas por guion (-) o guion bajo (_). Frontend: archivos en minúscula y sin espacios. Si son dos palabras, guión medio.

Tipo de archivo	Convención	Ejemplo	
Página principal	<code>index.html</code>	(debe llamarse así, es la predeterminada)	
Otras páginas HTML	minúsculas, guion medio	<code>about.html</code> , <code>contacto-form.html</code>	
CSS	minúsculas, guion medio	<code>styles.css</code> , <code>estilos-principales.css</code>	
JS	minúsculas, guion medio	<code>main.js</code> , <code>app.js</code> , <code>tareas-app.js</code>	
Imágenes	minúsculas, guion medio	<code>logo.png</code> , <code>foto-perfil.jpg</code>	
Carpetas	minúsculas, sin espacios	<code>css/</code> , <code>js/</code> , <code>images/</code> , <code>data/</code>	
Archivos JSON (si los usas)	minúsculas, guion medio	<code>tareas.json</code> , <code>usuarios.json</code>	

Contexto	Estilo recomendado	Ejemplo	
Carpetas (cualquier entorno)	minúsculas y guiones	<code>rutas</code> , <code>api</code> , <code>controladores</code>	
Archivos Node/Express	minúsculas con guion o subrayado	<code>servidor.js</code> , <code>tareas_controller.js</code>	
Componentes React	PascalCase	<code>ListaTareas.jsx</code> , <code>App.jsx</code>	
Funciones / Hooks React	camelCase	<code>useTareas.js</code> , <code>formatearFecha.js</code>	
CSS / imágenes / JSON	minúsculas	<code>styles.css</code> , <code>datos.json</code> , <code>logo.png</code>	

Vamos a abrir la carpeta back en la terminal y vamos a comenzar un proyecto en node “puro”. Ejecutamos: `npm init -y`

¿Qué ha hecho este comando?

Package.json

¿Qué es esto? El archivo package.json es el corazón de un proyecto Node.js.

Se crea automáticamente al ejecutar `npm init` o `npm init -y`.

El archivo package.json es el corazón de cualquier proyecto creado con Node.js. Contiene toda la información necesaria para que el entorno de desarrollo y otros programadores entiendan cómo funciona la aplicación. Se genera automáticamente al ejecutar el comando `npm init` o `npm init -y`, y en él se describen los datos básicos del proyecto, sus dependencias y los scripts que permiten ejecutarlo.

La propiedad "name" indica el nombre del proyecto; en este caso, backend. La propiedad "version" muestra la versión actual del proyecto, que comienza normalmente en 1.0.0 para señalar la primera versión estable. "description" sirve para escribir una breve explicación sobre el propósito o funcionalidad del proyecto, mientras que "main" señala el archivo principal que Node ejecutará por defecto al iniciar la aplicación (por ejemplo, index.js o server.js).

Dentro del apartado "scripts", se definen los comandos personalizados que pueden ejecutarse desde la terminal mediante `npm run`. Y también podemos definir comandos personalizados que nos ayudan al desarrollo. En este momento vamos a tocar el bloque script y lo vamos a modificar así. Son dos comandos básicos que facilitan el trabajo.

```
"scripts": {  
  "start": "node servidor.js",  
  "dev": "node --watch servidor.js"  
}
```

```
"start": "node servidor.js"
```

Ejecuta el servidor una sola vez.

Es el comando que se usa normalmente en producción o cuando ya no necesitas recargar el servidor cada vez que cambias algo.

```
"dev": "node --watch servidor.js"
```

Inicia el servidor en modo desarrollo.

La opción `--watch` le dice a Node que vigile el archivo y lo reinicie automáticamente cuando detecte cambios.

Esto evita tener que parar y volver a arrancar el servidor cada vez que modificas una línea de código.

Se ejecuta con: **npm run dev** solo tienes que ejecutar `npm run dev` una vez al empezar a trabajar.

No necesitas volver a escribir el comando tras cada cambio: Node se encarga de reiniciarlo por ti.

Las propiedades `"keywords"`, `"author"` y `"license"` son informativas: las dos primeras ayudan a identificar el proyecto y su autoría, y la última indica el tipo de licencia de uso, siendo ISC la predeterminada y una de las más abiertas.

Finalmente, para trabajar con la sintaxis moderna de módulos de JavaScript, es necesario añadir una última línea al archivo. Añade esta línea justo después de `"license": "ISC"`, para poder usar `import` y `export`.

```
"type": "module"
```

Esta instrucción le comunica a Node.js que se utilizará el formato ES Modules, lo que permite emplear las palabras clave `import` y `export` en lugar de `require()` y `module.exports`, facilitando un estilo de programación más actual y coherente con el estándar moderno del lenguaje.

Archivo json definitivo inicial

```
{  
  "name": "backend",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "start": "node servidor.js", --añadido  
    "dev": "node --watch servidor.js"---añadido  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "module" —añadido  
}
```

Fijate en la estructura:

Un sólo objeto principal encerrado entre llaves.

El archivo package.json es un documento en formato JSON (JavaScript Object Notation), y por tanto su contenido se organiza como un único objeto principal, encerrado entre llaves { }.

Dentro de esas llaves se declaran pares clave–valor, separados por comas.

Cada clave (como "name", "version", "scripts", etc.) representa una propiedad del proyecto, y su valor puede ser un texto, un número, un booleano, un array [] o incluso otro objeto { }.

- Las llaves externas { ... } contienen el objeto principal del proyecto.

- Dentro de él, "scripts" también tiene sus propias llaves, porque contiene otro objeto anidado con las órdenes personalizadas.
- El archivo JSON no puede tener más de un objeto principal, por lo que solo hay un par de llaves al principio y otro al final del archivo. No puedes poner comentarios
- Cada propiedad se separa por comas (,) salvo la última.

Si se añaden llaves o comas fuera de lugar, el archivo deja de ser válido y Node mostrará un error al intentar ejecutarlo. Debes tener cuidado cuando añades modificaciones.

Ejercicio 1: Conectamos un botón al back.

Trabajamos en el back.

Entonces ya tenemos dos carpetas una para el back y otra para el front. Ahora vamos a crear si no lo has hecho aún un archivo en cada una de ellas. En el back un archivo que se llame [servidor.js](#) y en el front un index. html

Pues ahora vamos a crear el servidor. ¿Recuerdas cómo era?

Paso. 1 Importa el módulo http.

Paso 2. Crea el servidor acuérdate de los dos parámetros básicos (req y res) .

```
const server = http.createServer((req, res) => {
```

Paso 3. Creamos una plantilla básica para manejar los CORS

En este paso estamos dentro de la función principal del servidor (es decir, dentro del `createServer()` o del `app.use()` en Express).

```
// CORS + JSON básicos (desarrollo)

res.setHeader("Access-Control-Allow-Origin", "*"); // permite cualquier origen
res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS");
res.setHeader("Access-Control-Allow-Headers", "Content-Type");
res.setHeader("Content-Type", "application/json; charset=utf-8");


// Responder al preflight
if (req.method === "OPTIONS") {
  res.writeHead(204);
  return res.end();
}
```

siempre dentro de la función.

Te voy a explicar esta plantilla pero tómalo a modo de información, cultura general digamos por que en cuanto pasemos a express aunque la lógica es la misma la sintaxis se ve reducida a un par de líneas.

Aquí añadimos una plantilla base que servirá para todos nuestros proyectos Node.js cuando necesitemos conectar un frontend (HTML, React, etc.) con un backend (Node) que está en otro puerto.

¿Por qué necesitamos esto? Cuando el frontend y el backend están en puertos distintos, el navegador bloquea las peticiones por seguridad.

Este sistema de protección se llama CORS (Cross-Origin Resource Sharing) y evita que una página web desconocida acceda a tus datos sin permiso.

Por ejemplo:

- Tu frontend puede estar en: `http://localhost:5173`
- Tu backend puede estar en: `http://localhost:3000`

El navegador ve que los "orígenes" son diferentes (5173 $\neq$ 3000) y pregunta: "¿El servidor de puerto 3000 da permiso para recibir peticiones desde otro origen?"

Si el servidor no responde correctamente, el navegador bloquea la conexión. Por eso añadimos estas cabeceras CORS.

Plantilla universal de cabeceras CORS (para desarrollo)

```
// CORS + JSON básicos (desarrollo)
```

```
res.setHeader("Access-Control-Allow-Origin", "*"); // Permite acceso desde cualquier origen
```

```
res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS"); // Métodos permitidos
```

```
res.setHeader("Access-Control-Allow-Headers", "Content-Type"); // Permite cabeceras personalizadas
```

```
res.setHeader("Content-Type", "application/json; charset=utf-8"); // Indicamos que enviamos JSON
```

```
// Responder al preflight (petición OPTIONS)
```

```
if (req.method === "OPTIONS") {
```

```
    res.writeHead(204); // Todo bien, pero no devuelvo nada
```

```
    return res.end();
```

```
}
```

Explicación paso a paso

- ♦ 1. Access-Control-Allow-Origin: *

Permite que cualquier sitio web (cualquier origen) pueda acceder a tu servidor.

En entornos reales NO se debe dejar el asterisco.

En producción se sustituye por la URL del frontend autorizado:

```
res.setHeader("Access-Control-Allow-Origin", "https://miweboficial.com");
```

♦ 2. Access-Control-Allow-Methods

Indica qué métodos HTTP acepta tu servidor.

Por ejemplo:

- GET → Leer información. Es el método que acepta por defecto.
 - POST → Enviar datos nuevos
 - PUT → Actualizar datos
 - DELETE → Eliminar datos
 - OPTIONS → Verificación previa (preflight) Sin esto, el navegador no permitiría usar PUT o DELETE. Es como una segunda verificación.
-

♦ 3. Access-Control-Allow-Headers

Permite usar cabeceras personalizadas (por ejemplo Content-Type o Authorization).

Esto es esencial cuando enviamos datos en formato JSON desde el frontend, porque el navegador incluye esa cabecera automáticamente:

```
headers: { "Content-Type": "application/json" }
```

- ♦ 4. Content-Type: application/json; charset=utf-8

Define el tipo de contenido que devuelve el servidor.

En este caso, JSON en formato UTF-8 (válido para cualquier idioma o símbolo).

Si no lo indicas, el navegador no sabrá cómo interpretar los datos.

- ♦ 5. Bloque OPTIONS

Cuando el navegador detecta que vas a hacer una petición “no simple” (por ejemplo POST con JSON o DELETE), envía primero una petición previa (preflight) con método OPTIONS para preguntar si el servidor permite esa operación.

El servidor debe responder con un código 204 (sin contenido) para confirmar que sí:

```
if (req.method === "OPTIONS") {  
  res.writeHead(204);  
  return res.end();  
}
```

Esto evita errores del tipo:

“Blocked by CORS policy: No ‘Access-Control-Allow-Origin’ header present...”

Añade esta plantilla al principio de cada uno de tus servidores en [Node.js](https://nodejs.org/)

Y cuando empieces con Express, verás que todo esto se resume en una sola línea: `app.use(cors());` // El servidor automáticamente permite los métodos PUT y DELETE, responde al preflight y añade las cabeceras necesarias.

4. Definimos rutas. Ya hemos creado el servidor y hemos definido las políticas CORS ahora vamos a definir nuestra ruta.

```
// Si la ruta es /api/saludo

if (req.url === '/api/saludo' && req.method === 'GET') {

  const respuesta = { mensaje: '¡Hola desde Node.js 🙌!', curso: 'UF1844' };

  res.writeHead(200);

  res.end(JSON.stringify(respuesta)); // enviamos texto JSON

}

});
```

La ruta que le vamos a dar al botón en el front.

5. Ponemos a escuchar al servidor.

```
server.listen(3000, () => {

  console.log('✅ Servidor en http://localhost:3000');

});
```

6. El control de errores.

Hemos creado un servidor que funciona pero no hemos tenido en cuenta la opción ¿y si no funciona? ¿te acuerdas como se controlaba esto?

Añade el bloque try...catch

En definitiva tu archivo servidor debe quedar así.

```
import http from "node:http";

const server = http.createServer((req, res) => {

  // 🧱 CORS + JSON básicos (desarrollo)

  res.setHeader("Access-Control-Allow-Origin", "*");

  res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS");

  res.setHeader("Access-Control-Allow-Headers", "Content-Type");

  res.setHeader("Content-Type", "application/json; charset=utf-8");

  // Responder a la pregunta de verificación

  if (req.method === "OPTIONS") {
```

Inma Contreras@2025

```
res.writeHead(204);

return res.end();
}

// Bloque de seguridad general
try {
  // Si la ruta es /api/saludo
  if (req.url === "/api/saludo" && req.method === "GET") {
    const respuesta = { mensaje: "¡Hola desde Node.js 🙌!", curso: "UF1844" };
    res.writeHead(200);
    return res.end(JSON.stringify(respuesta));
  }

  // Si la ruta no existe
  res.writeHead(404);
  res.end(JSON.stringify({ ok: false, error: "Ruta no encontrada" }));
}

catch (error) {
  // 🚨 Si ocurre un error inesperado, devolvemos un JSON de error
  console.error("❌ Error interno del servidor:", error);
  res.writeHead(500);
  res.end(JSON.stringify({ ok: false, error: "Error interno del servidor" }));
}

});

server.listen(3000, () => {
  console.log("✅ Servidor en http://localhost:3000");
});
```

Trabajamos en el front.

En tu carpeta frontend en el archivo index crea un botón.

Recuerda que el botón tendrá que tener una id para relacionarlo con el script y luego tendrás que crear un "hueco" un párrafo donde se pintará la respuesta.

Algo así.

```
<button id="boton">Cargar saludo</button>
<p id="resultado"></p>
```

Y ahora vamos con lo que realmente le da funcionalidad a tu botón y va a llamar al servidor. Ahora vamos a usar fetch.

Primero identificamos botón y párrafo.

```
<script>
  const boton = document.getElementById("boton");
  const resultado = document.getElementById("resultado");
```

Te recuerdo que hacía getElementById.

Busca en el documento un elemento cuyo atributo id="boton".Guarda la referencia en la constante boton.Si en el HTML no existe un elemento con id="boton", esto dará null y luego fallará al usarlo. Lo mismo para párrafo.

Ahora le decimos al botón que cuando reciba un click ejecute la siguiente función que le vamos a definir y en la que usaremos fetch porque queremos que llame a nuestro servidor. Por eso te decía que fetch es js del front.

```
boton.addEventListener("click", () => {
```

Y empezamos a definir la función. Usando fetch.

```
  fetch("http://localhost:3000/api/saludo")
```

Lanza una petición HTTP (por defecto GET) a esa URL.fetch devuelve una Promesa que se resolverá con un objeto Response cuando el servidor conteste.

localhost:3000 significa "mi propio ordenador en el puerto 3000"; tengo un servidor escuchando en este puerto así que nuestro servidor debe estar arrancado. Si el servidor no está activo, la promesa se rechaza (salta el .catch)

```
.then((res) => res.json())           // texto JSON → objeto JS
  .then((data) => {
    resultado.textContent = data.mensaje + " — " + data.curso;
  })
```

- Espera a que fetch se resuelva y recibe res, que es un Response (no es palabra reservada; podrías llamarlo respuesta).
- Llama a res.json(): esto lee el cuerpo de la respuesta y lo convierte de texto JSON a objeto de JavaScript.
- Ojo: res.json() también devuelve una Promesa; por eso encadenamos otro .then.

Cuando termina res.json(), recibimos el objeto data (por ejemplo { mensaje: "¡Hola...", curso: "UF1844" }).

Actualizamos el DOM: resultado.textContent cambia el texto visible dentro del elemento #resultado.

Concatenamos dos campos del objeto: data.mensaje y data.curso.

Recogemos los posibles errores

```
.catch((err) => {
  resultado.textContent = "❌ Error al conectar";
  console.error(err);
});
});
</script>
```

Captura **cualquier error** del proceso anterior:

- fallo de red (servidor apagado),
- URL mal escrita,
- error al convertir JSON (respuesta no era JSON),
- o un error lanzado manualmente.

Muestra un mensaje de error al usuario y loggea el detalle técnico en la consola del navegador. Es decir, no en el navegador si no en la consola que le damos al inspeccionar (F12). “Loguear” significa enviar mensajes a la consola del navegador para que el programador los vea, no el usuario. Es una herramienta de depuración, no de interfaz.

¿Te atreves a hacer otro botón? Prueba a hacer otro botón usando el mismo servidor pero que devuelva por ejemplo “Adios desde Node”.

Vamos a hacer algo muy parecido pero ahora vamos a intentar conectar una api externa.

Ejercicio 2: Conectando una API Externa.

[Enlace a git con código completo.](#)

Vamos a usar la [Giphy Api](#). Es una api sencilla de usar. ¿Cómo saber si una API es sencilla de usar? Buscamos la documentación y miramos las rutas disponibles y el ejemplo de respuestas. Tiene que devolver datos .json.

Conseguir tu API Key en developers.giphy.com

Inma Contreras@2025

Tener index.html, styles.css y script.js enlazados. Acuérdate de defer o llamar al script al final del body.

Paso 1. Configura el script.

// Clave personal de la API (GIPHY)

const API_KEY = "gggggggggggggggggggggggggggggggg"; //  tu clave real

const LIMIT = 8; // nº de GIFs por página

Paso 2. Crea en el HTML los campos que vamos a necesitar.

Elemento	ID o clase	Función principal
<code><form id="formulario-busqueda"></code>	formulario-busqueda	Contiene todo el buscador. El navegador detecta cuando el usuario pulsa "Buscar".
<code><input id="campo-busqueda"></code>	campo-busqueda	Es el cuadro de texto donde el usuario escribe la palabra que quiere buscar (por ejemplo, <i>gatos</i> o <i>memes</i>).
<code><button type="submit">Buscar</button></code>	—	Envía la búsqueda al script.
<code><button id="boton-tendencias">Tendencias</button></code>	boton-tendencias	Muestra los GIFs más populares del momento sin necesidad de escribir nada.
<code><p id="mensaje-estado"></code>	mensaje-estado	Muestra mensajes dinámicos: "Cargando...", "Sin resultados", "Error...", etc.
<code><section id="zona-resultados"></code>	zona-resultados	Es el contenedor donde se insertarán automáticamente los GIFs obtenidos de la API.

- El formulario recoge lo que el usuario escribe.
- El script hace una petición a la API con esa palabra.
- Los resultados se pintan dentro de la sección `<section id="zona-resultados">`.
- El mensaje de estado mantiene informada a la persona usuaria, incluso con lectores de pantalla.

Añade el estilo básico que necesites para darle una imagen profesional. Como mínimo tendrás que hacer un grid para que los resultados se pinten en cuadrículas. Y un formato a las casillas.

El estilo de este grid puede ser algo como así.

```
.rejilla{  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));  
  gap: 10px;  
}
```

/* Tarjeta mínima */

```
.card{  
  border: 1px solid #e5e7eb;  
  border-radius: 8px;  
  overflow: hidden;  
  background: #fff;  
}
```

/* Imagen de la tarjeta */

```
.card img{  
  display: block;  
  width: 100%;  
  height: 200px; /* altura fija simple */  
  object-fit: cover; /* recorta manteniendo proporción */  
}
```

/* Pie de la tarjeta (título + enlace) */

```
.card .info{  
  padding: 8px 10px;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;
```

```
gap: 8px;
font-size: 14px;
color: #111;
}

.card .title{
  margin: 0;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

Vamos a trabajar sobre el archivo js que como sabes es el que le da funcionalidad.

Este script tiene dos funciones una para pintar los resultados en el DOM y otra para conseguir los resultados.

Función 1.

```
function render(items) {
  if (!Array.isArray(items) || items.length === 0) {
    grid.innerHTML = `<p class="mini">Sin resultados. Prueba otra palabra.</p>`;
    return;
  }
  grid.innerHTML = items.map(it => {
    const src = it.images?.downsized_medium?.url || it.images?.original?.url;
    const titulo = it.title || "GIF";
    const link = it.url;
    return `
      <article class="card">
        
        <div class="info">
```

Inma Contreras@2025

```

    <h3 class="title" title="${titulo}">${titulo}</h3>

  </div>
</article>
`;
}).join("");
}

```

Explicación.

La función `render(items)` se encarga de mostrar en pantalla los resultados que llegan desde la API o desde una base de datos. Su función es transformar los datos en código HTML visible. Primero, comprueba que la variable `items` sea realmente un array y que contenga información; si no hay resultados, muestra un pequeño mensaje dentro del contenedor `grid` indicando que no se encontraron coincidencias. Si sí hay datos, utiliza el método `.map()` para recorrer cada elemento del array (cada objeto con la información de un GIF) y generar un bloque de HTML con una estructura de tarjeta: una imagen, un título y, opcionalmente, un enlace. Dentro del recorrido, usa el operador opcional `?.` para acceder de forma segura a las propiedades del objeto, evitando errores si alguna no existe, y el operador `||` para asignar valores por defecto (por ejemplo, mostrar "GIF" si no hay título). Una vez creadas todas las tarjetas, las une en un solo texto con `.join("")` y lo inserta en el elemento `grid` mediante `innerHTML`, lo que hace que aparezcan visualmente en la página. En resumen, `render()` no obtiene los datos, sino que pinta dinámicamente el contenido en el HTML a partir de la información ya recibida, convirtiendo los objetos JavaScript en elementos visuales accesibles y ordenados.

Ahora la segunda función. Usa `fetch` para conseguir los datos de la api.

```

// 🌐 Petición genérica
async function cargar(url) {
  try {

```

Inma Contreras@2025

```
estado.textContent = "Cargando...";

grid.setAttribute("aria-busy", "true");

grid.innerHTML = "";

const res = await fetch(url);
if (!res.ok) throw new Error("HTTP " + res.status);

const json = await res.json();
render(json.data);

const mostrados = Math.min(LIMIT, json.pagination?.count || json.data?.length || 0);
estado.textContent = `Listo. Mostrando ${mostrados} GIFs.`;
} catch (e) {
  console.error(e);
  estado.textContent = "❌ Error al cargar los GIFs.";
  grid.innerHTML = `<p class="mini">Revisa la consola para más detalles.</p>`;
} finally {
  grid.setAttribute("aria-busy", "false");
}
}
```

- async function cargar(url) declara una función asíncrona que recibe una url. Es asíncrona porque dentro usamos await para esperar respuestas de red sin bloquear la página.

Bloque try (flujo “feliz”)

1. Mensajes de estado y accesibilidad

- `estado.textContent = "Cargando..."`; informa al usuario (y lectores de pantalla) que empieza la carga.
- `grid.setAttribute("aria-busy", "true")`; marca el contenedor como ocupado para tecnologías asistivas.
- `grid.innerHTML = ""`; limpia resultados anteriores para evitar parpadeos o mezclar datos antiguos con nuevos.

2. Petición de red

- `const res = await fetch(url)`; hace la petición HTTP a la API (Giphy en nuestro caso) y espera la respuesta (Response).
- `if (!res.ok) throw new Error("HTTP " + res.status)`; valida el código HTTP: si no es 2xx, lanzamos un error manual (por ejemplo, 401, 404, 500). Esto envía la ejecución al catch.

3. Parseo y render

- `const json = await res.json()`; convierte el texto JSON de la respuesta en un objeto JavaScript.
- `render(json.data)`; pasamos el array de GIFs a la función de pintado. `render` no descarga nada; solo dibuja las tarjetas.

4. Mensaje final informativo

- `const mostrados = Math.min(LIMIT, json.pagination?.count || json.data?.length || 0);`
 - Calcula cuántos ítems se muestran:
 - Si la API incluye `pagination.count`, lo usa; si no, usa `json.data.length`.
 - Con `Math.min` lo limita a `LIMIT` (tu tope por página).
- `estado.textContent = `Listo. Mostrando ${mostrados} GIFs.`;`
comunica el resultado al usuario.

Bloque catch (errores)

- Si falla la red, la API devuelve error, o el `json()` da problemas, caemos aquí:
 - `console.error(e);` deja el detalle técnico en consola (útil para depurar).
 - `estado.textContent = "❌ Error al cargar los GIFs.";` mensaje claro para la persona usuaria.
 - `grid.innerHTML = `<p class="mini">Revisa la consola para más detalles.</p>`;` una pista visual en la zona de resultados.

Bloque finally (siempre se ejecuta)

- `grid.setAttribute("aria-busy", "false");` marca que el contenedor ya no está ocupado, tanto si todo fue bien como si hubo error. Es importante para

accesibilidad y consistencia del estado de la UI.

Resumen.

- Preparar la UI (decir “cargando”, limpiar, marcar ocupado) →
- Pedir datos (fetch) y validar HTTP →
- Convertir JSON y pintar (render) →
- Informar cuántos elementos se muestran →
- Capturar errores con un mensaje amable →
- Restaurar accesibilidad con `aria-busy=false`. Es la etiqueta que avisa si se han cargado los resultados o no. False significa que la carga ha terminado y el contenido es estable, por lo que ahora sí puede leerse o anunciarse al usuario.

Ahora nos queda el final ya: **conectar la lógica del JavaScript con la interacción del usuario** —es decir, las que hacen que **tu aplicación cobre vida**.

// 📦 URLs rápidas

```
const urlBuscar = q =>
`https://api.giphy.com/v1/gifs/search?api_key=${API_KEY}&q=${encodeURIComponent
(q)}&limit=${LIMIT}&rating=pg`;

const urlTendencias =
`https://api.giphy.com/v1/gifs/trending?api_key=${API_KEY}&limit=${LIMIT}&rating=pg
`;
```

// 🖱️ Eventos

```
form.addEventListener("submit", e => {
  e.preventDefault();

  const q = input.value.trim();

  if (!q) { estado.textContent = "Escribe una palabra para buscar."; return; }

  cargar(urlBuscar(q));
```

Inma Contreras@2025


```
});
```

```
btnTop.addEventListener("click", () => {  
  input.value = "";  
  cargar(urlTendencias);  
});
```

```
// 🚀 Al cargar la página: tendencias
```

```
window.addEventListener("DOMContentLoaded", () => cargar(urlTendencias));
```

URLs rápidas (las “direcciones” de la API). Son funciones que construyen la URL completa que necesita `fetch()` para pedir datos a Giphy.

- `urlBuscar(q)` → genera la dirección para buscar un término concreto (por ejemplo, “cats” o “memes”).
 - `encodeURIComponent(q)` se asegura de que, si hay espacios o tildes, la URL siga siendo válida.
- `urlTendencias` → es una dirección fija que pide los GIFs en tendencia (los más vistos del momento).

Cada vez que el usuario busca algo o pulsa el botón de tendencias, tu script las usa para saber a qué endpoint debe llamar la función `cargar()`.

Eventos: conectar el HTML con la lógica JS. `addEventListener("submit")`: escucha cuando el usuario pulsa el botón Buscar o presiona Enter en el formulario.

`e.preventDefault()`: evita que el navegador recargue la página (comportamiento por defecto del formulario).

`input.value.trim()`: recoge el texto del campo de búsqueda, quitando espacios innecesarios.

Si el campo está vacío → muestra un mensaje.

Si hay texto → llama a `cargar(urlBuscar(q))`, es decir, descarga y muestra los GIFs de esa búsqueda.

Botón de Tendencias. Escucha cuando se pulsa el botón Tendencias.

Limpia el campo de texto (`input.value = ""`).

Llama a `cargar(urlTendencias)` para mostrar los GIFs más populares.

Cargar tendencias al abrir la página

- `DOMContentLoaded` se ejecuta cuando el HTML ya está listo (antes incluso de que se carguen todas las imágenes).
- Llamamos a `cargar(urlTendencias)` para que, nada más abrir la página, aparezcan automáticamente algunos GIFs populares.

Conclusión Node.js (UF1844)

Node.js es un entorno de ejecución de código abierto, multiplataforma, que permite a los desarrolladores crear aplicaciones del lado del servidor utilizando

JavaScript puro, funcionando fuera del contexto de un navegador. Su característica más importante es su naturaleza no bloqueante, diseñada para potenciar la programación asíncrona. Node.js usa el motor V8 de Google y se basa en el Event Loop para manejar tareas lentas (como operaciones de red o de archivos) en segundo plano, sin detener el hilo principal de ejecución.

El desarrollo en Node.js se fundamenta en el uso de módulos, que pueden ser: propios (archivos JavaScript reutilizables creados por el desarrollador), de terceros (gestionados por NPM), o nativos (integrados en el núcleo de Node.js).

1. Módulos Nativos Fundamentales: fs y http

Los módulos nativos son las "herramientas básicas" de Node.js, listas para usar sin necesidad de instalación adicional. Dos de los más cruciales son:

A. fs (File System)

El módulo fs permite interactuar con el sistema operativo para manejar archivos. Es esencial para operaciones como:

- Leer archivos (fs.readFile).
- Escribir nuevos archivos (fs.writeFile). Esto se usa para guardar logs, manejar plantillas o generar archivos dinámicos en el servidor.
- Copiar, renombrar o eliminar archivos.
- En el contexto de la conversación previa, este módulo sería el mecanismo que usaría el servidor para "guardar" la información de las tarjetas generadas si el cliente la enviara al backend.

B. http

El módulo http es el corazón del backend en Node.js, ya que permite la creación de servidores web capaces de escuchar peticiones del navegador y enviar respuestas.

- Gracias a http, el ordenador puede funcionar como un servidor real, recibiendo una petición (req) del cliente (navegador) y enviando una respuesta (res).

- El método `createServer()` recibe una función de callback que se ejecuta con cada petición HTTP.
- Es la base para construir APIs, donde el servidor analiza la URL y el método (GET, POST) y devuelve datos, normalmente en formato JSON.
- En proyectos donde el frontend y el backend usan diferentes puertos, el módulo `http` (o frameworks contruidos sobre él) debe gestionar las cabeceras CORS para prevenir el bloqueo de seguridad por parte del navegador.

2. Programación Asíncrona: Promesas, `async` y `await`

La capacidad de Node.js de no ser bloqueante se maneja a través de mecanismos asíncronos. Históricamente, esto se manejaba con callbacks, pero esto llevó al problema conocido como "Callback Hell". La solución moderna son las Promesas.

A. Las Promesas (Promise)

Una Promesa es un objeto que representa un valor que puede estar disponible ahora, en el futuro o nunca. El estándar moderno permite encadenar tareas y manejar errores de forma limpia.

- Se construyen con `new Promise((resolve, reject) => { ... })`.
- Devuelven dos posibles valores: `resolve()` (éxito) o `reject()` (error).
- Se consumen utilizando los métodos nativos del objeto Promise: `.then()` (captura el éxito/resolve), `.catch()` (captura el fracaso/reject) y `.finally()` (se ejecuta siempre).

B. `async` y `await`

Introducidas en ES8 (2017), `async` y `await` simplifican la escritura de código asíncrono, haciéndolo parecer secuencial y más legible. Estas palabras clave trabajan exclusivamente con Promesas.

- `async`: Se coloca delante de una función para indicar que esta devolverá automáticamente un objeto Promesa.

- `await`: Solo puede usarse dentro de una función `async` (o al nivel superior en módulos ES modernos). Pausa la ejecución del código dentro de esa función hasta que la Promesa se resuelva, consumiéndola implícitamente sin necesidad de `.then()`.
- El control de errores con `await` se maneja mediante el bloque `try...catch`. Si la Promesa es rechazada (`reject`), el control se transfiere al bloque `catch`.

3. Comunicación de Red: `fetch()`

`fetch()` es la función nativa de JavaScript (tanto en el navegador como en Node.js moderno) que actúa como el puente de comunicación entre el frontend y el backend (APIs o servidores propios).

- Se usa para realizar peticiones HTTP (GET, POST, etc.) y siempre devuelve una Promesa que se resuelve con un objeto `Response`.
- Para leer los datos devueltos por el servidor (típicamente JSON), es necesario encadenar una segunda llamada asíncrona: `res.json()`, la cual también devuelve una Promesa que convierte el texto JSON en un objeto JavaScript operable (`data`).
- Al ser una función que devuelve una Promesa, se recomienda usar la sintaxis `async/await` junto con `try...catch` para manejar la petición y el parseo de datos de manera clara y robusta.

En resumen, el desarrollo robusto en Node.js se apoya en el uso eficiente de los módulos nativos (`fs` para el disco, `http` para la red) y la gestión moderna de la asincronía (`Promise`, `async/await`) para interactuar con el sistema y comunicarse con el cliente mediante herramientas como `fetch()`.

